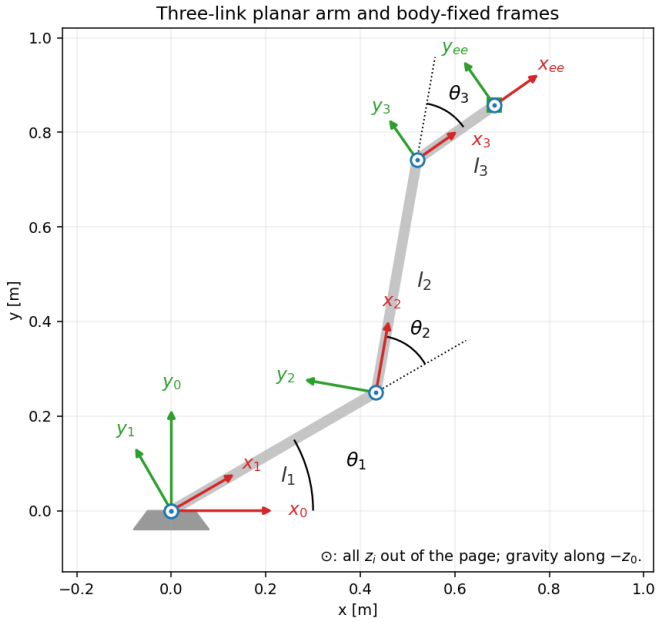
ECE276C Assignment 1

Robot Kinematics and Simple Control

Sachin Bhadang
PID: A69040722
Email: sbhadang@ucsd.edu

1. Arm and body-fixed coordinate frames



Frame $\{0\}$ is fixed at the base. Frame $\{i\}$ is attached to link i at joint i, with x_i along the link and z_i along the joint axis. Frame $\{ee\}$ is at the tip, with the same orientation as $\{3\}$. All frames are right-handed. The offsets follow assignment Table 1: $a_0 = 0$, $a_1 = l_1$, $a_2 = l_2$, $a_3 = l_3$.

2. End-effector kinematics

Let $\mathbf{q} = [\theta_1, \theta_2, \theta_3]^\top$ and $\mathbf{x} = f(\mathbf{q}) = [X, Y, \theta]^\top$. Then

$$
\begin{aligned}
X &= l_1 \cos\theta_1 + l_2 \cos(\theta_1 + \theta_2) + l_3 \cos(\theta_1 + \theta_2 + \theta_3), \\
Y &= l_1 \sin\theta_1 + l_2 \sin(\theta_1 + \theta_2) + l_3 \sin(\theta_1 + \theta_2 + \theta_3), \\
\theta &= \theta_1 + \theta_2 + \theta_3 \pmod{2\pi}.
\end{aligned}
$$

At $\mathbf{q} = \mathbf{0}$, the arm points along $+x_0$ and the tip is at $(l_1 + l_2 + l_3, 0)$.

3. Required PyBullet configurations

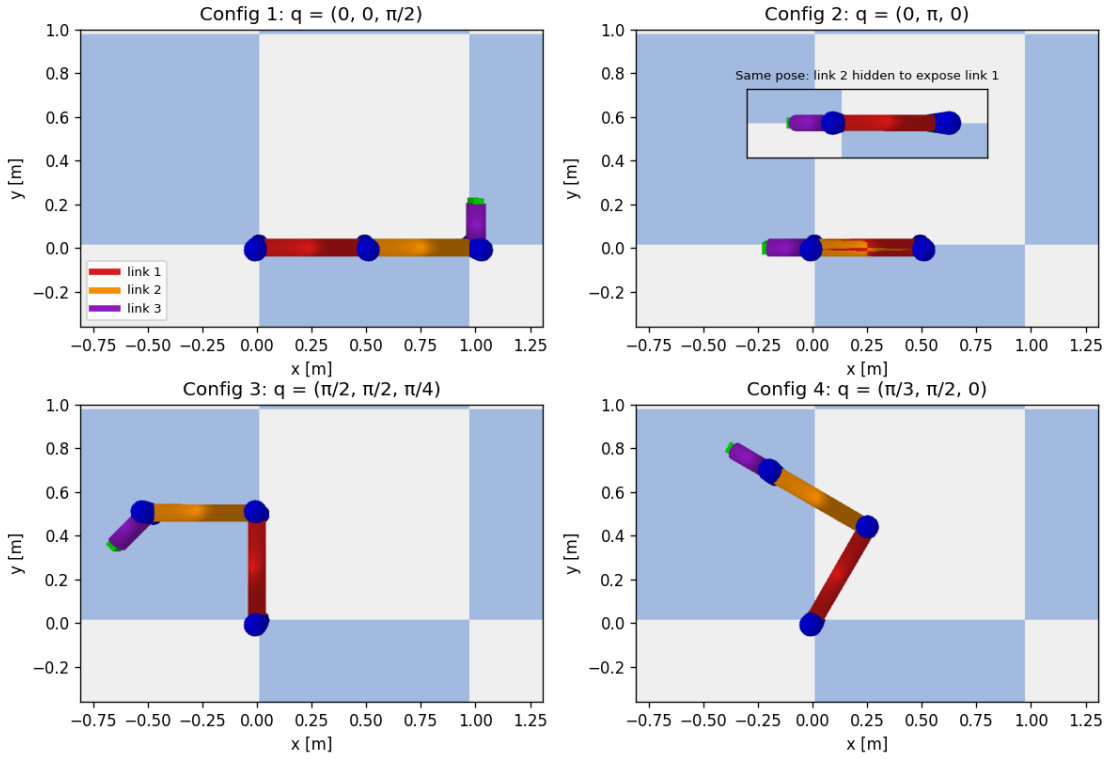
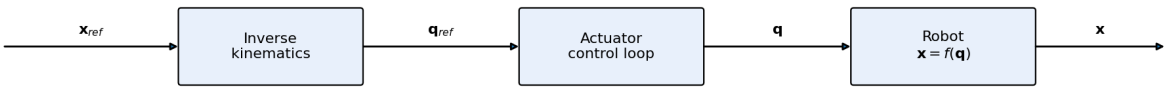


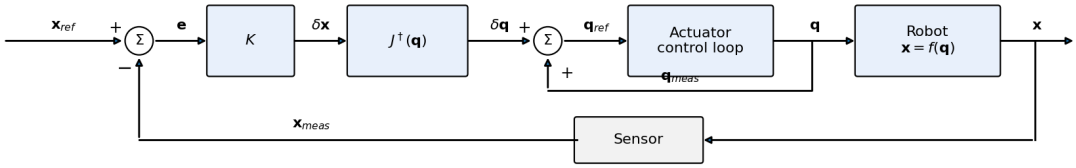
Figure 1: The supplied URDF with $l_1 = l_2 = 0.5\,\text{m}$ and $l_3 = 0.2\,\text{m}$. Links 1, 2 and 3 are red, orange and purple. In configuration 2, links 1 and 2 coincide exactly; the labeled inset hides link 2 to expose link 1 without changing the joint configuration.

4. Open-loop and closed-loop control diagrams

(a) Open-loop setpoint control



(b) Closed-loop setpoint control



Both diagrams use $\mathbf{x}_{ref} = [X, Y, \theta]^{\top}_{ref}$. In (b), $J = \partial f / \partial \mathbf{q} \in \mathbb{R}^{3\times 3}$, $\mathbf{e} = \mathbf{x}_{ref} - \mathbf{x}_{meas}$ (orientation error wrapped to $[-\pi, \pi)$), and $\mathbf{q}_{ref} = \mathbf{q}_{meas} + \delta\mathbf{q}$.

5. Smooth profile and open-loop motion

For a move of duration T, let $\tau = t/T$ and use

$$\mathbf{x}_{ref}(t) = \mathbf{x}_0 + s(\tau)(\mathbf{x}_f - \mathbf{x}_0), \qquad s(\tau) = 35\tau^4 - 84\tau^5 + 70\tau^6 - 20\tau^7.$$

The polynomial satisfies the boundary conditions

$$s(0) = 0, \quad s(1) = 1, \quad s'(0) = s'(1) = 0, \quad s''(0) = s''(1) = 0, \quad s'''(0) = s'''(1) = 0,$$

where primes denote derivatives with respect to τ. In physical time,

$$\dot{s} = \frac{140\tau^3(1-\tau)^3}{T}, \qquad \ddot{s} = \frac{420\tau^2(1-\tau)^2(1-2\tau)}{T^2}, \qquad \dddot{s} = \frac{840\tau(1-\tau)(1-5\tau+5\tau^2)}{T^3}.$$

The profile is smooth in the second-order sense: s is thrice continuously differentiable (C^3), so velocity and acceleration have continuous derivatives, including at the waypoints, where velocity, acceleration and jerk are all zero. All coordinates use the same s, so each Cartesian segment is straight.

The four moves $1 \to 2 \to 3 \to 4 \to 1$ take 2.5 s each, totaling 10 s; $\theta_{ref} = 0$ throughout. At 240 Hz, analytic inverse kinematics converts each sampled pose into joint setpoints on one continuous elbow branch. These drive PyBullet's position motors, with joint velocity feedforward; measured end-effector pose is used only for recording.

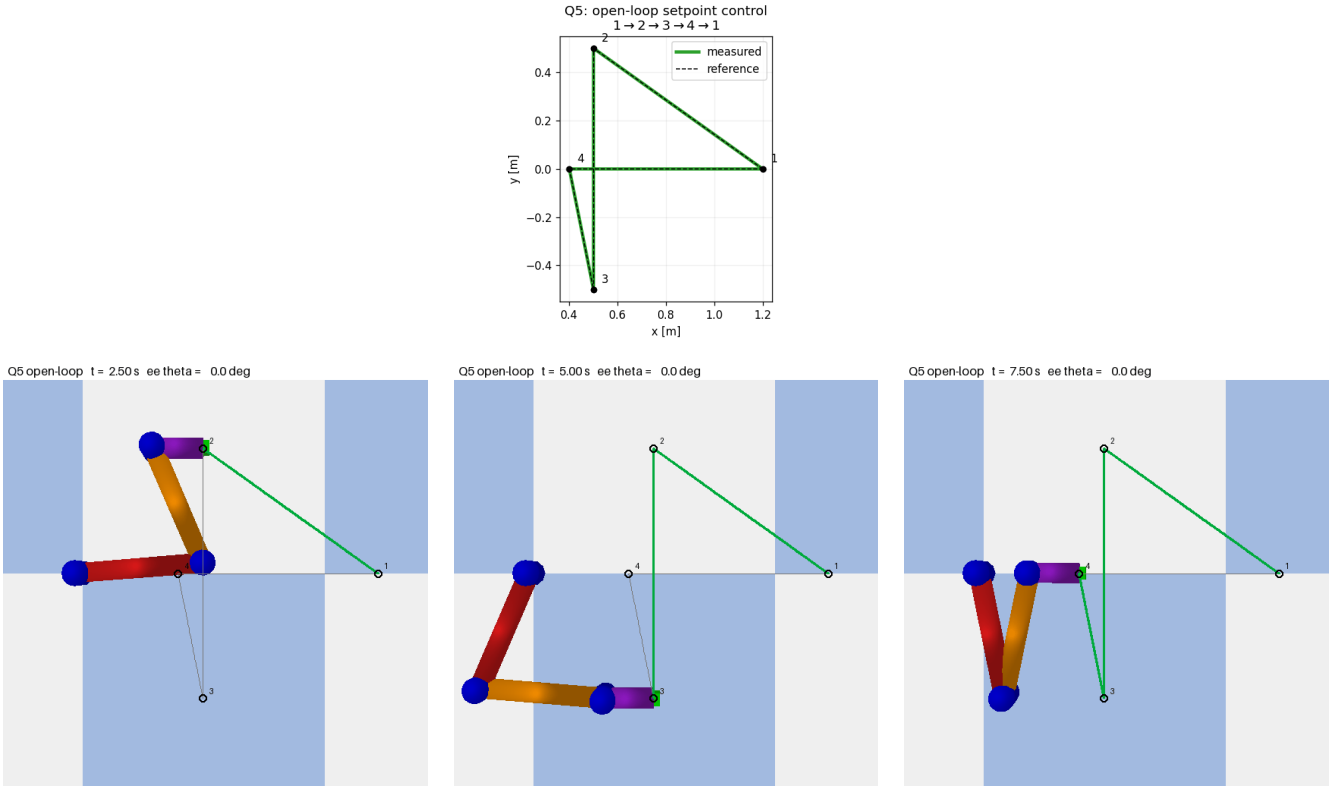


Figure 2: Q5: measured end-effector path (top) and animation frames at $t = 2.5$, 5.0 and 7.5 s (bottom, left to right), reaching locations 2, 3 and 4 with $\theta = 0$.

Animation: `q5_open_loop.gif` (20 fps, 10.05 s including the final frame). The reference velocity is zero at every waypoint. In the sampled simulation, waypoint speeds are below 3×10^{-5} m/s and the maximum position error is 3.9 μm.

6. Closed-loop motion and nullspace control

The task is now $\mathbf{x}_{xy} = [X, Y]^\top$; θ is free. Use the same reference path and timing as Q5, with $J_{xy} = \partial \mathbf{x}_{xy}/\partial \mathbf{q} \in \mathbb{R}^{2\times 3}$. Compute the SVD pseudoinverse $J_{xy}^\dagger$ and

$$N = I_3 - J_{xy}^\dagger J_{xy}, \qquad J_{xy} N = 0.$$

Define the interior angles at the shared joints by

$$\alpha_{12} = \pi - |\bar{\theta}_2|, \qquad \alpha_{23} = \pi - |\bar{\theta}_3|, \qquad \bar{\theta}_i = \mathrm{wrap}_{[-\pi,\pi)}(\theta_i).$$

A straight pair of links has the largest interior angle, π. To drive both angles toward this value subject to the position constraint, minimize their squared departures from π:

$$U(\mathbf{q}) = \tfrac{1}{2}\left[(\pi - \alpha_{12})^2 + (\pi - \alpha_{23})^2\right] = \tfrac{1}{2}(\bar{\theta}_2^2 + \bar{\theta}_3^2), \qquad \nabla_{\mathbf{q}} U = [0, \bar{\theta}_2, \bar{\theta}_3]^\top.$$

The angles stay away from the wrap discontinuity during this motion. Use projected gradient descent:

$$\dot{\mathbf{q}}_2 = -\lambda \nabla_{\mathbf{q}} U, \qquad \dot{\mathbf{q}}_{null} = N \dot{\mathbf{q}}_2, \qquad \lambda = 10\,\mathrm{s}^{-1}.$$

Combining it with the next-sample closed-loop trajectory step gives

$$\delta \mathbf{q} = J_{xy}^\dagger(\mathbf{x}_{xy,ref}[k+1] - \mathbf{x}_{xy,meas}[k]) + \Delta t\, \dot{\mathbf{q}}_{null}, \qquad \Delta t = 1/240\,\mathrm{s}.$$

Thus $J_{xy}\dot{\mathbf{q}}_{null} = 0$: the secondary controller changes posture without adding Cartesian velocity. This is a local optimization of U along the position constraint.

The implementation reads the joint angles and end-effector position each step. It corrects the finite-step linearization error with repeated Jacobian position corrections, then sends the resulting joint increment divided by Δt to the velocity motors. At location 1 the arm is straight and J_{xy} is singular; analytic IK selects the first nonzero departure sample, and the final sample returns to the straight posture. The simulated initial state is exactly $\mathbf{q} = \mathbf{0}$.

Animation: `q6_closed_loop.gif` (20 fps, 10.05 s). The maximum measured position error is below 2 μm, and waypoint speeds are below 4×10^{-5} m/s. The nullspace identity is exact in the kinematic model; the simulation follows the prescribed path to this numerical accuracy.

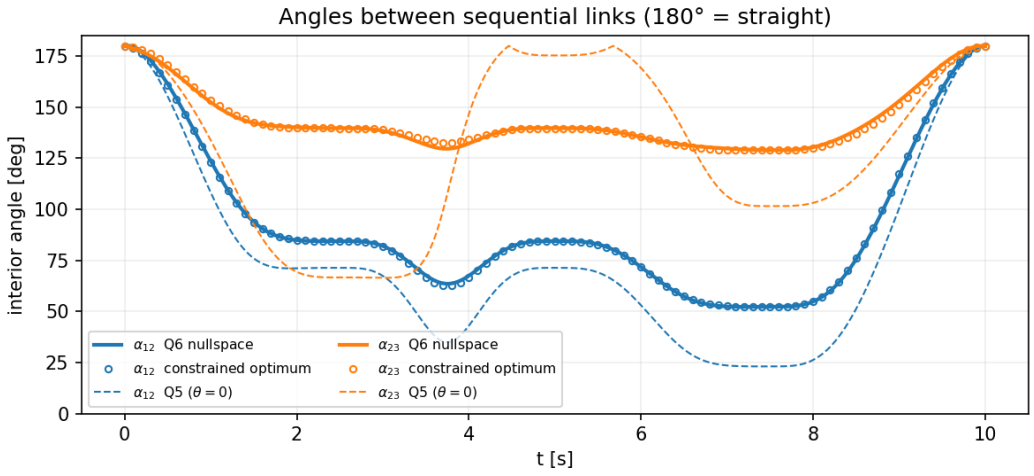


Figure 3: Interior angles between sequential links. Solid: Q6 nullspace controller. Circles: constrained optimum, found by brute force over every end-effector orientation and both elbow branches that reach the same (X, Y) reference. Dashed: the fixed-$\theta = 0$ posture of Q5.

Figure 3 checks the secondary objective. The nullspace controller stays within 1.3° (α_{12}) and 3.1° (α_{23}) of the constrained optimum along the whole path; the small lag comes from the finite gain λ. Its smallest interior angles are 52° and 129°, against 23° and 66° for Q5's fixed-orientation posture.

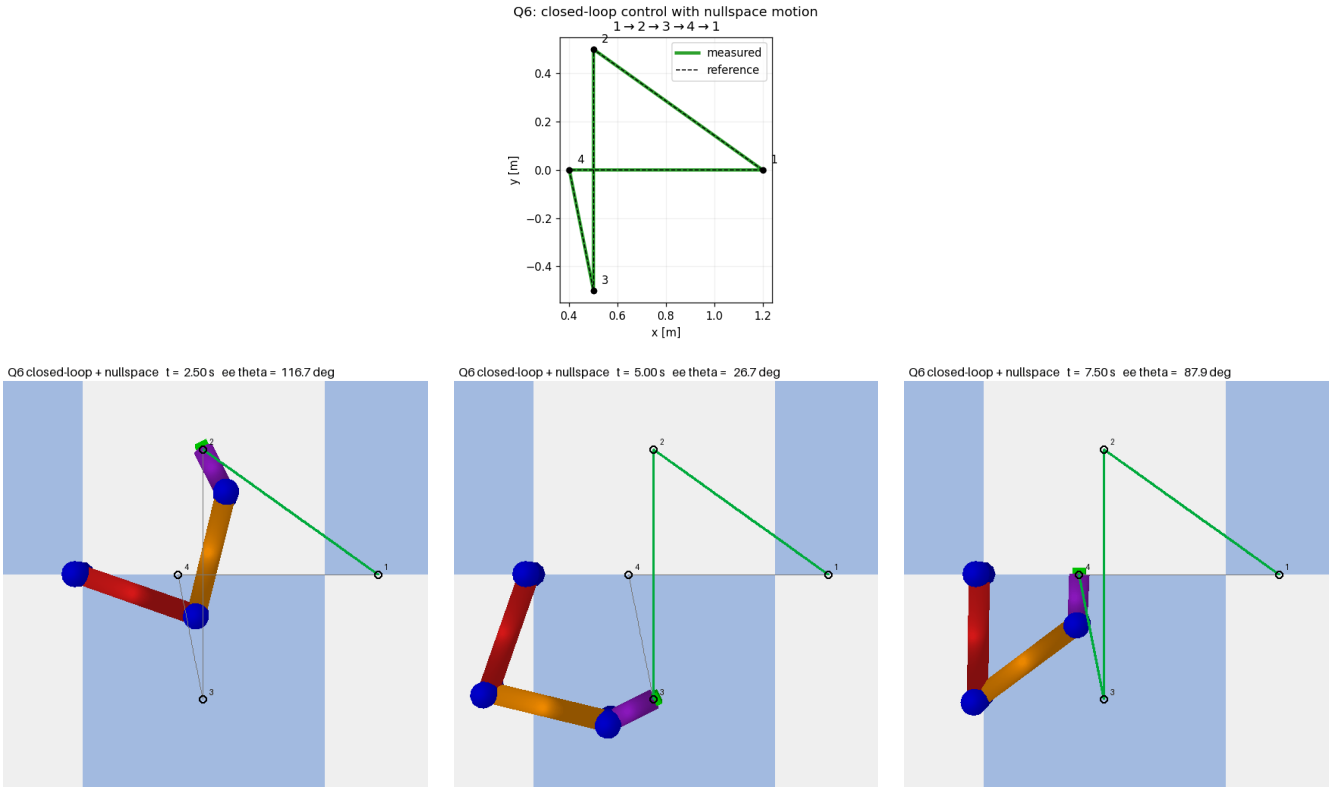


Figure 4: Q6: measured end-effector path (top) and animation frames at $t = 2.5$, 5.0 and 7.5 s (bottom, left to right). The same locations are reached while orientation is free.

Code

The code uses Python 3 with numpy, matplotlib, pybullet, imageio and Pillow, and sits in one folder next to the supplied `three-link-robot.urdf`. `python a1_solution.py` runs Questions 3, 5 and 6 and writes the renders, GIFs, path plots and logged data to `results/`; `python figures.py` then draws the Question 1 and 4 figures, the GIF snapshots and the Question 6 link-angle comparison from those results. Together they take about 45 s.

arm.py: kinematics

Forward kinematics, analytic Jacobian, closed-form inverse kinematics, and the SVD pseudoinverse with its nullspace projector.

```python
"""Kinematics of the planar three-link arm (l1 = l2 = 0.5 m, l3 = 0.2 m)."""
import numpy as np

L = np.array([0.5, 0.5, 0.2])


def wrap(a):
    """Wrap angle(s) to [-pi, pi)."""
    return (np.asarray(a) + np.pi) % (2 * np.pi) - np.pi


def fk(q, L=L):
    """End-effector pose [x, y, theta] from joint angles q = [q1, q2, q3].

    x     = l1 c1 + l2 c12 + l3 c123
    y     = l1 s1 + l2 s12 + l3 s123
    theta = q1 + q2 + q3
    """
    c = np.cumsum(q)
    return np.array([np.sum(L * np.cos(c)), np.sum(L * np.sin(c)), c[-1]])


def joint_points(q, L=L):
    """Planar positions of joint 1, joint 2, joint 3 and the end effector (4x2)."""
    c = np.cumsum(q)
    seg = np.stack([L * np.cos(c), L * np.sin(c)], axis=1)
    return np.vstack([[0.0, 0.0], np.cumsum(seg, axis=0)])


def jacobian(q, L=L):
    """3x3 Jacobian d[x, y, theta]/dq. Rows 0:2 are the position Jacobian."""
    c = np.cumsum(q)
    J = np.ones((3, 3))
    for i in range(3):
        J[0, i] = -np.sum(L[i:] * np.sin(c[i:]))
        J[1, i] = np.sum(L[i:] * np.cos(c[i:]))
    return J


def ik(pose, elbow=1.0, q_near=None, L=L):
    """Analytic inverse kinematics for pose [x, y, theta].

    The wrist point (x, y) - l3 [cos theta, sin theta] is solved as a 2-link arm.
    elbow = +1 / -1 picks the elbow branch. If q_near is given, each angle is
    shifted by multiples of 2 pi to the value closest to q_near (continuity).
    """
    x, y, th = pose
    xw, yw = x - L[2] * np.cos(th), y - L[2] * np.sin(th)
    c2 = (xw**2 + yw**2 - L[0]**2 - L[1]**2) / (2 * L[0] * L[1])
    if abs(c2) > 1.0 + 1e-10:
        raise ValueError(f'Unreachable end-effector pose: {pose}')
    q2 = elbow * np.arccos(np.clip(c2, -1.0, 1.0))
    q1 = np.arctan2(yw, xw) - np.arctan2(L[1] * np.sin(q2), L[0] + L[1] * np.cos(q2))
```

```
    q = np.array([q1, q2, th - q1 - q2])
    if q_near is not None:
        q = q_near + wrap(q - q_near)
    return q


def pinv_and_null(J):
    """SVD pseudoinverse and orthogonal nullspace projector (Lecture 4.5).

    Damping must not be used in I - J^dagger J: it would leak the secondary
    motion into the Cartesian task. Retain all nonzero singular values.
    """
    u, s, vt = np.linalg.svd(J, full_matrices=True)
    tol = np.finfo(float).eps * max(J.shape) * s[0]
    rank = int(np.count_nonzero(s > tol))
    J_dagger = (vt[:rank].T / s[:rank]) @ u[:, :rank].T
    basis = vt[rank:].T
    return J_dagger, basis @ basis.T
```

a1_solution.py: simulation and control

PyBullet setup and top-down rendering, the Question 3 configurations, the C^3 trajectory profile, Question 5 open-loop setpoint control, and Question 6 closed-loop control with the nullspace term.

```
"""ECE276C Assignment 1 -- Robot Kinematics and Simple Control.

Usage (Python with numpy, matplotlib, pybullet, imageio and Pillow):
    python a1_solution.py            # run Q3, Q5, Q6 headless, write results/
    python a1_solution.py --part 5   # one part only
    python a1_solution.py --gui      # also open the PyBullet GUI while simulating
"""
import argparse
import os

import imageio.v2 as imageio
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import numpy as np
import pybullet as p
import pybullet_data
from PIL import Image, ImageDraw, ImageFont

from arm import fk, ik, jacobian, pinv_and_null, wrap

HERE = os.path.dirname(os.path.abspath(__file__))
OUT = os.path.join(HERE, 'results')
DT = 1. / 240.

CONFIGS = np.array([[0, 0, np.pi / 2],
                    [0, np.pi, 0],
                    [np.pi / 2, np.pi / 2, np.pi / 4],
                    [np.pi / 3, np.pi / 2, 0]])
LOCATIONS = np.array([[1.2, 0.0, 0.0],
                      [0.5, 0.5, 0.0],
                      [0.5, -0.5, 0.0],
                      [0.4, 0.0, 0.0],
                      [1.2, 0.0, 0.0]])    # 1 -> 2 -> 3 -> 4 -> 1
TOTAL_TIME = 10.0
GIF_FPS = 20
FONT = ImageFont.load_default(size=16)


# ---------------------------------------------------------------------------- simulation
def make_sim(gui=False):
    """Same setup as the starter code; returns robot id, movable joints, ee link."""
    if p.isConnected():
        p.disconnect()
```

```
    p.connect(p.GUI if gui else p.DIRECT)
    p.setAdditionalSearchPath(pybullet_data.getDataPath())
    p.setGravity(0., 0., -9.81)
    p.setTimeStep(DT)
    # lowered slightly so the arm (centred on z = 0) is not half buried in the floor
    p.loadURDF('plane.urdf', [0, 0, -0.06])
    robot = p.loadURDF(os.path.join(HERE, 'three-link-robot.urdf'), useFixedBase=True)

    movable_joints, ee_link = [], None
    for idx in range(p.getNumJoints(robot)):
        info = p.getJointInfo(robot, idx)
        if info[2] != p.JOINT_FIXED:
            movable_joints.append(idx)
        if info[12].decode('utf-8') == 'end_effector':
            ee_link = idx
    # distinct colours per link so overlapping links can be told apart
    for joint, rgba in zip(movable_joints, [[0.85, 0.1, 0.1, 1], [0.95, 0.55, 0.0, 1],
                                            [0.55, 0.1, 0.75, 1]]):
        p.changeVisualShape(robot, joint, rgbaColor=rgba)
    # disable the default velocity motors so we fully own the joints
    p.setJointMotorControlArray(robot, movable_joints, p.VELOCITY_CONTROL, forces=[0.] * 3)
    return robot, movable_joints, ee_link


def set_q(robot, joints, q):
    for j, qi in zip(joints, q):
        p.resetJointState(robot, j, qi, 0.)


def get_q(robot, joints):
    states = p.getJointStates(robot, joints)
    return np.array([s[0] for s in states]), np.array([s[1] for s in states])


def ee_pose(robot, ee_link):
    """Measured end-effector [x, y, theta] from PyBullet."""
    st = p.getLinkState(robot, ee_link, computeForwardKinematics=True)
    yaw = p.getEulerFromQuaternion(st[5])[2]
    return np.array([st[4][0], st[4][1], yaw])


class TopCamera:
    """Top-down camera. The z = 0 plane maps affinely to pixels, so world xy
    points can be drawn directly onto the rendered image."""

    def __init__(self, center, half_height, size=(560, 560), fov=50.):
        self.W, self.H = size
        self.c = np.asarray(center, float)
        height = half_height / np.tan(np.radians(fov / 2))
        self.hh = half_height
        self.hw = half_height * self.W / self.H
        self.view = p.computeViewMatrix([self.c[0], self.c[1], height],
                                        [self.c[0], self.c[1], 0.], [0, 1, 0])
        self.proj = p.computeProjectionMatrixFOV(fov, self.W / self.H, 0.05, height + 2)

    def render(self):
        img = p.getCameraImage(self.W, self.H, self.view, self.proj,
                               renderer=p.ER_TINY_RENDERER, shadow=0)[2]
        return np.reshape(img, (self.H, self.W, 4))[:, :, :3].astype(np.uint8)

    def to_px(self, xy):
        xy = np.atleast_2d(xy)
        u = (xy[:, 0] - self.c[0]) / self.hw * self.W / 2 + self.W / 2
        v = self.H / 2 - (xy[:, 1] - self.c[1]) / self.hh * self.H / 2
        return list(zip(u.tolist(), v.tolist()))

    def extent(self):
        return [self.c[0] - self.hw, self.c[0] + self.hw, self.c[1] - self.hh, self.c[1] + self.hh]


# ---------------------------------------------------------------------------- Q3
```

```python
def part3(gui=False):
    robot, joints, ee = make_sim(gui)
    cam = TopCamera(center=[0.25, 0.32], half_height=0.68, size=(700, 450))
    fig, axes = plt.subplots(2, 2, figsize=(10, 6.8))
    print('\nQ3: FK check (analytic vs PyBullet)')
    for k, (q, ax) in enumerate(zip(CONFIGS, axes.ravel()), start=1):
        set_q(robot, joints, q)
        img = cam.render()
        imageio.imwrite(os.path.join(OUT, f'q3_config{k}.png'), img)

        pose_fk, pose_pb = fk(q), ee_pose(robot, ee)
        print(f'  config {k}: q = {np.round(q, 3)}  fk = {np.round(pose_fk, 4)}  '
              f'pybullet = {np.round(pose_pb, 4)}  '
              f'|dpos| = {np.linalg.norm(pose_fk[:2] - pose_pb[:2]):.1e}  '
              f'dtheta = {abs(wrap(pose_fk[2] - pose_pb[2])):.1e}')

        ax.imshow(img, extent=cam.extent())
        ax.set_title(f'Config {k}: q = ({_pi_str(q[0])}, {_pi_str(q[1])}, {_pi_str(q[2])})')
        ax.set_xlabel('x [m]')
        ax.set_ylabel('y [m]')
        if k == 2:
            # Links 1 and 2 coincide exactly in this configuration. Keep the
            # unchanged full render and add a labelled cutaway of the same pose.
            p.changeVisualShape(robot, joints[1], rgbaColor=[0.95, 0.55, 0., 0.])
            cutaway_cam = TopCamera([0.15, 0.], 0.17, size=(600, 170))
            cutaway = cutaway_cam.render()
            p.changeVisualShape(robot, joints[1], rgbaColor=[0.95, 0.55, 0., 1.])
            imageio.imwrite(os.path.join(OUT, 'q3_config2_cutaway.png'), cutaway)
            inset = ax.inset_axes([0.08, 0.57, 0.84, 0.23])
            inset.imshow(cutaway)
            inset.set_title('Same pose: link 2 hidden to expose link 1', fontsize=8)
            inset.set_xticks([])
            inset.set_yticks([])
    axes[0, 0].plot([], [], color=(0.85, 0.1, 0.1), lw=4, label='link 1')
    axes[0, 0].plot([], [], color=(0.95, 0.55, 0.0), lw=4, label='link 2')
    axes[0, 0].plot([], [], color=(0.55, 0.1, 0.75), lw=4, label='link 3')
    axes[0, 0].legend(loc='lower left', fontsize=8)
    fig.tight_layout()
    fig.savefig(os.path.join(OUT, 'q3_configs.png'), dpi=120)
    plt.close(fig)
    p.disconnect()


def _pi_str(a):
    names = {0: '0', np.pi: 'π', np.pi / 2: 'π/2', np.pi / 3: 'π/3', np.pi / 4: 'π/4'}
    return next((s for v, s in names.items() if np.isclose(a, v)), f'{a:.2f}')


# ---------------------------------------------------------------------------- trajectory
def segment_durations():
    """Four equal moves, with each waypoint on a simulation time step."""
    return np.full(len(LOCATIONS) - 1, TOTAL_TIME / (len(LOCATIONS) - 1))


def profile(tau):
    """C3 move-and-stop profile; velocity and acceleration have continuous derivatives.

    Lecture 4's polynomial boundary-value construction with zero endpoint
    velocity, acceleration and jerk: s = 35*tau^4 - 84*tau^5 + 70*tau^6 - 20*tau^7.
    Returns s, s', s''.
    """
    tau = np.clip(tau, 0., 1.)
    s = tau**4 * (35. + tau * (-84. + tau * (70. - 20. * tau)))
    ds = 140. * tau**3 * (1. - tau)**3
    dds = 420. * tau**2 * (1. - tau)**2 * (1. - 2. * tau)
    return s, ds, dds


def reference(t, durations=None):
    """Straight-line reference pose and velocity at time t."""
    durations = segment_durations() if durations is None else durations
```

```
    starts = np.concatenate([[0.], np.cumsum(durations)])
    i = int(np.clip(np.searchsorted(starts, t, side='right') - 1, 0, len(durations) - 1))
    T = durations[i]
    s, ds, _ = profile(np.clip((t - starts[i]) / T, 0., 1.))
    a, b = LOCATIONS[i], LOCATIONS[i + 1]
    return a + s * (b - a), ds / T * (b - a)


# ---------------------------------------------------------------------------- GIF helpers
def draw_frame(img, cam, desired_xy, actual_xy, t, title, extra=''):
    im = Image.fromarray(img)
    d = ImageDraw.Draw(im)
    d.line(cam.to_px(desired_xy), fill=(120, 120, 120), width=1)
    if len(actual_xy) > 1:
        d.line(cam.to_px(actual_xy), fill=(0, 170, 60), width=3)
    for k, (u, v) in enumerate(cam.to_px(LOCATIONS[:4, :2]), start=1):
        d.ellipse([u - 5, v - 5, u + 5, v + 5], outline=(0, 0, 0), width=2)
        d.text((u + 8, v - 16), str(k), fill=(0, 0, 0))
    d.rectangle([0, 0, cam.W, 24], fill=(255, 255, 255))
    d.text((6, 3), f'{title}   t = {t:5.2f} s   {extra}', fill=(0, 0, 0), font=FONT)
    return np.asarray(im)


def desired_path(n=400):
    ts = np.linspace(0, TOTAL_TIME, n)
    return np.array([reference(t)[0][:2] for t in ts])


def run_and_record(step_fn, robot, joints, ee, title, gif_name):
    """Shared loop: call step_fn(t) each physics step, log data, write GIF."""
    cam = TopCamera(center=[0.55, 0.0], half_height=0.85)
    n_steps = int(round(TOTAL_TIME / DT))
    every = int(round(1. / (GIF_FPS * DT)))
    log = {k: [] for k in ['t', 'x_des', 'x_meas', 'q', 'qdot']}
    frames, des_xy = [], desired_path()
    for k in range(n_steps + 1):
        t = k * DT
        x_des, _ = reference(t)
        x_meas = ee_pose(robot, ee)
        q, qdot = get_q(robot, joints)
        for key, val in zip(log, [t, x_des, x_meas, q, qdot]):
            log[key].append(val)
        if k % every == 0:
            act = np.array(log['x_meas'])[:, :2]
            frames.append(draw_frame(cam.render(), cam, des_xy, act, t, title,
                                     f'ee theta = {np.degrees(wrap(x_meas[2])):6.1f} deg'))
        if k < n_steps:
            step_fn(t, q, x_meas)
            p.stepSimulation()
    imageio.mimsave(os.path.join(OUT, gif_name), frames, duration=1000. / GIF_FPS, loop=0)
    log = {k: np.array(v) for k, v in log.items()}
    np.savez_compressed(os.path.join(OUT, gif_name.replace('.gif', '.npz')), **log)
    return log


def plot_results(log, fname, title):
    err = np.linalg.norm(log['x_meas'][:, :2] - log['x_des'][:, :2], axis=1)
    fig, ax = plt.subplots(figsize=(6, 4.5))
    ax.plot(log['x_meas'][:, 0], log['x_meas'][:, 1], color='tab:green', lw=3, label='measured')
    ax.plot(log['x_des'][:, 0], log['x_des'][:, 1], 'k--', lw=1, label='reference')
    for k, (x, y, _) in enumerate(LOCATIONS[:4], start=1):
        ax.plot(x, y, 'ko', ms=5)
        ax.annotate(str(k), (x, y), xytext=(7, 6), textcoords='offset points')
    ax.set_aspect('equal')
    ax.set_xlabel('x [m]')
    ax.set_ylabel('y [m]')
    ax.set_title(title + '\n' + r'$1\to2\to3\to4\to1$')
    ax.legend(loc='upper right')
    ax.grid(alpha=0.2)
    fig.tight_layout()
    fig.savefig(os.path.join(OUT, fname), dpi=120)
```

```
    plt.close(fig)
    return err


# ------------------------------------------------------------------------ Q5
def part5(gui=False):
    """Open-loop setpoint control: the reference pose is converted to joint
    setpoints with analytic IK; the end-effector pose is never fed back."""
    robot, joints, ee = make_sim(gui)
    q_prev = ik(LOCATIONS[0], elbow=1.)
    set_q(robot, joints, q_prev)

    def step(t, q_meas, x_meas):
        nonlocal q_prev
        x_des, _ = reference(t)
        q_des = ik(x_des, elbow=1., q_near=q_prev)
        q_next = ik(reference(t + DT)[0], elbow=1., q_near=q_des)
        p.setJointMotorControlArray(robot, joints, p.POSITION_CONTROL,
                                    targetPositions=q_des.tolist(),
                                    targetVelocities=((q_next - q_des) / DT).tolist(),
                                    positionGains=[0.5] * 3, velocityGains=[1.] * 3,
                                    forces=[200.] * 3)
        q_prev = q_des

    log = run_and_record(step, robot, joints, ee, 'Q5 open-loop', 'q5_open_loop.gif')
    err = plot_results(log, 'q5_open_loop.png', 'Q5: open-loop setpoint control')
    th_err = np.abs(wrap(log['x_meas'][:, 2] - log['x_des'][:, 2]))
    print(f'\nQ5 open loop: max |pos err| = {err.max():.2e} m, '
          f'RMS = {np.sqrt(np.mean(err**2)):.2e} m, max |θ err| = {np.degrees(th_err.max()):.2e} deg')
    _print_setpoints(log)
    p.disconnect()
    return log


def _print_setpoints(log):
    starts = np.concatenate([[0.], np.cumsum(segment_durations())])
    for k, ts in enumerate(starts):
        i = int(round(ts / DT))
        speed = np.linalg.norm(jacobian(log['q'][i])[:2] @ log['qdot'][i])
        print(f'  t = {ts:5.2f} s  location {[1, 2, 3, 4, 1][k]}: '
              f'measured ee = {np.round(log["x_meas"][i], 4)}, speed = {speed:.2e} m/s')


# ------------------------------------------------------------------------ Q6
K_NULL = 10.     # nullspace gradient gain


def null_gradient(q):
    """Gradient of U = 1/2 (wrap(q2)^2 + wrap(q3)^2), away from wrap cuts.

    The interior angle at a shared joint is alpha_i = pi - |wrap(q_{i+1})|.
    U penalises squared departures from the largest angle, pi. This is a local
    posture objective, not a claim of a global maximum of both angles.
    """
    qw = wrap(q)
    return np.array([0., qw[1], qw[2]])


def closed_loop_target(q, x_meas, t):
    """Lecture 4's measured-state, next-sample resolved-rate step plus nullspace.

    Correct the O(dt^2) error of the linearised step by repeated Cartesian
    corrections. No simulated joint state is reset during motion.
    """
    x_next, _ = reference(t + DT)
    if t == 0.:
        # At full extension J has rank one. IK selects the departure branch at
        # the first nonzero sample; the actual initial state is exactly q = 0.
        return ik(x_next, elbow=1., q_near=q)
    if t + DT >= TOTAL_TIME - 1e-12:
        # The final full-reach point has the unique straight-arm posture.
```

```
        return q + wrap(-q)
    J_dagger, N = pinv_and_null(jacobian(q)[:2])
    delta_x = x_next[:2] - x_meas[:2]
    q_next = q + J_dagger @ delta_x - DT * K_NULL * N @ null_gradient(q)
    # Retain measured Cartesian feedback while correcting finite-step error.
    goal = fk(q)[:2] + delta_x
    for _ in range(12):
        error = goal - fk(q_next)[:2]
        if np.linalg.norm(error) < 1e-10:
            break
        J_dagger, _ = pinv_and_null(jacobian(q_next)[:2])
        correction = J_dagger @ error
        # Limit Newton updates near full extension without changing the goal.
        q_next += correction * min(1., 0.2 / max(np.linalg.norm(correction), 1e-15))
    return q_next


def part6(gui=False):
    """Closed-loop position tracking with projected gradient descent on U(q)."""
    robot, joints, ee = make_sim(gui)
    set_q(robot, joints, np.zeros(3))

    def step(t, q_meas, x_meas):
        q_next = closed_loop_target(q_meas, x_meas, t)
        qdot = (q_next - q_meas) / DT
        p.setJointMotorControlArray(robot, joints, p.VELOCITY_CONTROL,
                                    targetVelocities=qdot.tolist(), forces=[500.] * 3)

    log = run_and_record(step, robot, joints, ee, 'Q6 closed-loop + nullspace', 'q6_closed_loop.gif')
    err = plot_results(log, 'q6_closed_loop.png', 'Q6: closed-loop control with nullspace motion')
    print(f'Q6 closed loop: max |pos err| = {err.max() * 1e3:.3f} mm, '
          f'RMS = {np.sqrt(np.mean(err**2)) * 1e3:.3f} mm')
    _print_setpoints(log)
    p.disconnect()
    return log


# ---------------------------------------------------------------------------
if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('--part', type=int, choices=[3, 5, 6])
    parser.add_argument('--gui', action='store_true')
    args = parser.parse_args()
    os.makedirs(OUT, exist_ok=True)
    print('segment durations [s]:', np.round(segment_durations(), 3))
    for part, fn in [(3, part3), (5, part5), (6, part6)]:
        if args.part in (None, part):
            fn(args.gui)
```

figures.py: report figures

The Question 1 frame diagram, the Question 4 block diagrams, the animation snapshots, and the Question 6 comparison with the brute-force optimum.

```
"""Report figures and representative frames extracted from the Q5/Q6 GIFs."""
import os

import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.patches import Arc, FancyBboxPatch
from PIL import Image

from arm import L, joint_points, wrap

OUT = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'results')

```

```python

# ---------------------------------------------------------------------------- Q1
def frame(ax, origin, angle, name, length=0.16, z_mark=True, x_label_shift=(0., 0.)):
    """Draw x (red) and y (green) axes of a planar frame; z points out of the page."""
    o = np.asarray(origin)
    for vec_angle, color, lbl, shift in [(angle, 'tab:red', 'x', x_label_shift),
                                         (angle + np.pi / 2, 'tab:green', 'y', (0., 0.))]:
        d = length * np.array([np.cos(vec_angle), np.sin(vec_angle)])
        ax.annotate('', o + d, o, arrowprops=dict(arrowstyle='-|>', color=color, lw=1.8))
        ax.text(*(o + 1.22 * d + np.asarray(shift)), f'${lbl}_{{{name}}}$', color=color, fontsize=12,
                ha='center', va='center')
    if z_mark:
        ax.plot(*o, 'o', ms=9, mfc='w', mec='tab:blue', mew=1.5, zorder=5)
        ax.plot(*o, '.', ms=4, color='tab:blue', zorder=6)


def angle_arc(ax, center, a0, a1, r, label):
    lo, hi = sorted([a0, a1])
    ax.add_patch(Arc(center, 2 * r, 2 * r, theta1=np.degrees(lo), theta2=np.degrees(hi), lw=1.2))
    mid = (a0 + a1) / 2
    ax.text(center[0] + 1.35 * r * np.cos(mid), center[1] + 1.35 * r * np.sin(mid), label,
            fontsize=13, ha='center', va='center')


def q1_figure():
    q = np.radians([30., 50., -45.])
    pts = joint_points(q)
    c = np.cumsum(q)
    fig, ax = plt.subplots(figsize=(8, 6))
    ax.plot(pts[:, 0], pts[:, 1], '-', color='0.35', lw=7, solid_capstyle='round', alpha=0.35, zorder=1)
    ax.plot(pts[:-1, 0], pts[:-1, 1], 'ko', ms=7, zorder=2)
    ax.plot(*pts[-1], 's', color='tab:green', ms=9, zorder=2)

    # base and world frame {0}
    ax.fill([-0.08, 0.08, 0.05, -0.05], [-0.04, -0.04, 0.0, 0.0], color='0.6', zorder=0)
    frame(ax, [0, 0], 0., '0', length=0.22)
    frame(ax, pts[0], c[0], '1')
    frame(ax, pts[1], c[1], '2')
    # x3 label below link 3, clear of the theta3 arc
    frame(ax, pts[2], c[2], '3', length=0.11, x_label_shift=0.045 * np.array([np.sin(c[2]), -np.cos(c[2])]))
    frame(ax, pts[3], c[2], 'ee', length=0.12)

    # joint angles: measured from the previous x axis (dashed extension) to the new one
    angle_arc(ax, pts[0], 0., c[0], 0.3, r'$\theta_1$')
    for i, (a_prev, a_new) in enumerate([(c[0], c[1]), (c[1], c[2])], start=2):
        o = pts[i - 1]
        ext = o + 0.22 * np.array([np.cos(a_prev), np.sin(a_prev)])
        ax.plot([o[0], ext[0]], [o[1], ext[1]], 'k:', lw=1)
        angle_arc(ax, o, a_prev, a_new, 0.12, fr'$\theta_{i}$')

    for i in range(3):
        m = (pts[i] + pts[i + 1]) / 2
        nrm = np.array([-np.sin(c[i]), np.cos(c[i])])
        ax.text(*(m - (0.09 if i == 2 else 0.06) * nrm), f'$l_{i + 1}$', fontsize=14, ha='center', va='center',
     ↪ color='0.2')

    ax.text(0.98, 0.02, r'$\odot$: all $z_i$ out of the page; gravity along $-z_0$.',
            transform=ax.transAxes, ha='right', fontsize=10)
    ax.set_aspect('equal')
    ax.set_xlim(-0.23, 1.02)
    ax.set_ylim(-0.13, 1.02)
    ax.set_xlabel('x [m]')
    ax.set_ylabel('y [m]')
    ax.set_title('Three-link planar arm and body-fixed frames')
    ax.grid(alpha=0.2)
    fig.tight_layout()
    fig.savefig(os.path.join(OUT, 'q1_frames.png'), dpi=140)
    plt.close(fig)


# ---------------------------------------------------------------------------- Q4
```

```python
def box(ax, xy, text, w=1.7, h=0.8, fc='#e8f0fb'):
    x, y = xy
    ax.add_patch(FancyBboxPatch((x - w / 2, y - h / 2), w, h, boxstyle='round,pad=0.03', fc=fc, ec='k'))
    ax.text(x, y, text, ha='center', va='center', fontsize=10)
    return dict(l=(x - w / 2, y), r=(x + w / 2, y), t=(x, y + h / 2), b=(x, y - h / 2))


def summer(ax, xy, r=0.17):
    ax.add_patch(plt.Circle(xy, r, fc='w', ec='k'))
    ax.text(*xy, r'$\Sigma$', ha='center', va='center', fontsize=10)
    x, y = xy
    return dict(l=(x - r, y), r=(x + r, y), t=(x, y + r), b=(x, y - r))


def arrow(ax, a, b, label=None, off=(0, 0.14), sign=None, sign_off=(-0.12, 0.1)):
    ax.annotate('', b, a, arrowprops=dict(arrowstyle='-|>', lw=1.3))
    if label:
        ax.text((a[0] + b[0]) / 2 + off[0], (a[1] + b[1]) / 2 + off[1], label, ha='center', fontsize=10)
    if sign:
        ax.text(b[0] + sign_off[0], b[1] + sign_off[1], sign, fontsize=11, ha='center')


def poly(ax, pts, end_arrow=True):
    pts = np.array(pts)
    ax.plot(pts[:-1, 0], pts[:-1, 1], 'k', lw=1.3)
    if end_arrow:
        ax.annotate('', pts[-1], pts[-2], arrowprops=dict(arrowstyle='-|>', lw=1.3))


def q4_figure():
    """Full-pose setpoint control, following Lecture 4 slides 10 and 14."""
    fig, (a1, a2) = plt.subplots(2, 1, figsize=(12, 4.7))
    for ax in (a1, a2):
        ax.set_xlim(-0.2, 13.5)
        ax.set_aspect('equal')
        ax.axis('off')

    a1.set_ylim(-0.8, 1.)
    a1.set_title('(a) Open-loop setpoint control', fontsize=12, loc='left')
    inv = box(a1, (3., 0.), 'Inverse\nkinematics', w=2.)
    act = box(a1, (6.8, 0.), 'Actuator\ncontrol loop', w=2.)
    rob = box(a1, (10.5, 0.), 'Robot\n' r'$\mathbf{x}=f(\mathbf{q})$', w=2.)
    arrow(a1, (0., 0.), inv['l'], r'$\mathbf{x}_{ref}$')
    arrow(a1, inv['r'], act['l'], r'$\mathbf{q}_{ref}$')
    arrow(a1, act['r'], rob['l'], r'$\mathbf{q}$')
    arrow(a1, rob['r'], (13., 0.), r'$\mathbf{x}$')

    a2.set_ylim(-1.65, 1.)
    a2.set_title('(b) Closed-loop setpoint control', fontsize=12, loc='left')
    summ = summer(a2, (1.65, 0.))
    gain = box(a2, (3.0, 0.), r'$K$', w=1.0)
    pinv = box(a2, (4.9, 0.), r'$J^\dagger(\mathbf{q})$', w=1.4)
    summ_q = summer(a2, (6.6, 0.))
    act = box(a2, (8.45, 0.), 'Actuator\ncontrol loop', w=1.7)
    rob = box(a2, (11.2, 0.), 'Robot\n' r'$\mathbf{x}=f(\mathbf{q})$', w=1.7)
    arrow(a2, (0., 0.), summ['l'], r'$\mathbf{x}_{ref}$', sign='+')
    arrow(a2, summ['r'], gain['l'], r'$\mathbf{e}$')
    arrow(a2, gain['r'], pinv['l'], r'$\delta\mathbf{x}$')
    arrow(a2, pinv['r'], summ_q['l'], r'$\delta\mathbf{q}$', sign='+')
    arrow(a2, summ_q['r'], act['l'], r'$\mathbf{q}_{ref}$')
    arrow(a2, act['r'], rob['l'], r'$\mathbf{q}$')
    arrow(a2, rob['r'], (13., 0.), r'$\mathbf{x}$')
    # measured joint angles: q_ref = q_meas + delta q
    poly(a2, [(9.8, 0.), (9.8, -0.6), (6.6, -0.6), summ_q['b']])
    a2.text(6.75, -0.45, '+', fontsize=11)
    a2.text(8.2, -0.47, r'$\mathbf{q}_{meas}$', fontsize=10, ha='center')
    sensor = box(a2, (7.7, -1.2), 'Sensor', w=1.5, h=0.5, fc='#f2f2f2')
    poly(a2, [(12.5, 0.), (12.5, -1.2), sensor['r']])
    poly(a2, [sensor['l'], (1.65, -1.2), summ['b']])
    a2.text(1.38, -0.4, '−', fontsize=13)
    a2.text(3.5, -1.05, r'$\mathbf{x}_{meas}$', fontsize=10)
```

```
    fig.tight_layout(h_pad=1.5)
    fig.savefig(os.path.join(OUT, 'q4_block_diagrams.png'), dpi=160)
    plt.close(fig)


def animation_snapshots():
    """Extract the actual GIF frames at the three intermediate waypoints."""
    times_ms = [2500, 5000, 7500]
    for stem in ['q5_open_loop', 'q6_closed_loop']:
        with Image.open(os.path.join(OUT, stem + '.gif')) as gif:
            elapsed_ms = 0
            target = 0
            for index in range(gif.n_frames):
                gif.seek(index)
                duration_ms = gif.info['duration']
                if elapsed_ms <= times_ms[target] < elapsed_ms + duration_ms:
                    gif.convert('RGB').save(os.path.join(OUT, f'{stem}_frame{target + 1}.png'))
                    target += 1
                    if target == len(times_ms):
                        break
                elapsed_ms += duration_ms


# ------------------------------------------------------------------------- Q6
def optimal_link_angles(p_xy, n=3600):
    """Brute-force reference for Q6: over every ee orientation that reaches p_xy
    (both elbow branches), the posture minimising wrap(q2)^2 + wrap(q3)^2.
    Returns the two interior angles [rad]."""
    th = np.linspace(-np.pi, np.pi, n, endpoint=False)
    xw, yw = p_xy[0] - L[2] * np.cos(th), p_xy[1] - L[2] * np.sin(th)
    c2 = (xw**2 + yw**2 - L[0]**2 - L[1]**2) / (2 * L[0] * L[1])
    best, best_q = np.inf, None
    for elbow in (1., -1.):
        q2 = elbow * np.arccos(np.clip(c2, -1., 1.))
        q1 = np.arctan2(yw, xw) - np.arctan2(L[1] * np.sin(q2), L[0] + L[1] * np.cos(q2))
        q3 = th - q1 - q2
        cost = wrap(q2)**2 + wrap(q3)**2
        cost[np.abs(c2) > 1. + 1e-9] = np.inf
        i = np.argmin(cost)
        if cost[i] < best:
            best, best_q = cost[i], (q2[i], q3[i])
    return np.pi - np.abs(wrap(np.array(best_q)))


def q6_link_angles():
    """Interior angles of Q6 vs the brute-force optimum and the theta = 0 posture of Q5."""
    q6 = np.load(os.path.join(OUT, 'q6_closed_loop.npz'))
    q5 = np.load(os.path.join(OUT, 'q5_open_loop.npz'))
    t = q6['t']
    a6 = np.pi - np.abs(wrap(q6['q'][:, 1:]))
    a5 = np.pi - np.abs(wrap(q5['q'][:, 1:]))
    sub = np.arange(0, len(t), 24)
    opt = np.array([optimal_link_angles(x[:2]) for x in q6['x_des'][sub]])

    fig, ax = plt.subplots(figsize=(8, 3.8))
    for j, color, name in [(0, 'tab:blue', r'$\alpha_{12}$'), (1, 'tab:orange', r'$\alpha_{23}$')]:
        ax.plot(t, np.degrees(a6[:, j]), color=color, lw=2, label=f'{name}  Q6 nullspace')
        ax.plot(t[sub], np.degrees(opt[:, j]), 'o', color=color, ms=3.5, mfc='none',
                label=f'{name}  constrained optimum')
        ax.plot(t, np.degrees(a5[:, j]), '--', color=color, lw=1, label=f'{name}  Q5 ($\\theta=0$)')
    ax.set_xlabel('t [s]')
    ax.set_ylabel('interior angle [deg]')
    ax.set_ylim(0, 185)
    ax.set_title('Angles between sequential links (180° = straight)')
    ax.legend(fontsize=7.5, ncol=2, loc='lower left')
    ax.grid(alpha=0.2)
    fig.tight_layout()
    fig.savefig(os.path.join(OUT, 'q6_link_angles.png'), dpi=150)
    plt.close(fig)

    gap = np.degrees(np.abs(a6[sub] - opt)).max(axis=0)
```

```
    print(f'Q6 min interior angles: {np.degrees(a6.min(axis=0)).round(1)} deg; '
          f'Q5 (theta = 0): {np.degrees(a5.min(axis=0)).round(1)} deg; '
          f'max deviation from optimum: {gap.round(2)} deg')


if __name__ == '__main__':
    os.makedirs(OUT, exist_ok=True)
    q1_figure()
    q4_figure()
    animation_snapshots()
    q6_link_angles()
```